Goal and Evidence Continuity in Adaptive Agent Swarms

Runtime contracts and an evaluation protocol for Pegasus

James Paul Jackson

Version 1.1 · 8 November 2026

Abstract

Adaptive agent systems can change their task structure, execution owners, and model providers while pursuing a user goal. Those changes create a continuity problem: an unfinished requirement can lose its owner, evidence can become inapplicable after an artifact revision, and an interrupted operation can be repeated without a known outcome. This paper specifies a runtime contract for preserving accepted goal obligations and the applicability of completion evidence across such changes. The design separates a versioned goal contract from the execution graph, treats agents as replaceable executors, and mediates authoritative transitions through a deterministic acceptance kernel. It combines durable state, ownership epochs, evidence dependency tracking, and reconciliation of uncertain effects. Two conditional propositions characterize structural continuity and semantic soundness; neither establishes correctness for arbitrary natural-language goals. A proposed admission policy also accounts for the verification and integration work created by additional agents. An evaluation protocol separates deterministic fault injection from native model experiments and compares the mechanisms under matched budgets. Pegasus provides the implementation setting. The paper presents a design and research protocol, not measured performance improvements or a machine-checked proof. Its intended contribution is a precise, testable composition of established mechanisms and a reproducible method for evaluating goal continuity in adaptive swarms.

Keywords. Adaptive agents; durable execution; goal contracts; evidence applicability; recursive delegation; fault injection; verification scheduling.

Contents

- 1 Introduction
- 2 Related work and research position
- 2.1 Goals and verifiable delegation
- 2.2 Durable runtimes and evidence management
- 2.3 Scope of the proposed contribution
- 3 System model and assumptions
- 3.1 Persistent goals and replaceable execution
- 3.2 Evidence applicability
- 3.3 Trust and failure assumptions
- 4 Runtime architecture
- 5 Continuity protocol
- 5.1 Graph mutation and delegation

- 5.2 Result acceptance and artifact changes
- 5.3 Failure recovery and external effects
- 5.4 Provider handoff and contract amendment
- 6 Conditional correctness properties
- 6.1 Structural continuity
- 6.2 Semantic soundness under stronger assumptions
- 6.3 Progress and impossibility boundaries
- 7 Admission based on verification demand
- 8 Implementation setting in Pegasus
- 9 Evaluation protocol
- 9.1 Questions and experimental layers
- 9.2 Fault scenarios and independent oracles
- 9.3 Baselines and ablations
- 9.4 Metrics and statistical analysis
- 10 Limitations and research risks
- 11 Conclusion
- Appendix A. Minimum reproducibility record
- References

1 Introduction

An agent may complete its assignment without completing the user's goal. A group of agents may finish every item on its task list while omitting a requirement that was never assigned. A recovered workflow may faithfully replay a recorded decision even though the artifact supporting that decision has changed. These cases concern the relationship between the goal, the current execution state, and the evidence used to accept the result.

The problem becomes more visible as agents acquire the ability to create children, exchange messages, change plans, and invoke tools. A useful adaptive system must permit this flexibility while retaining an authoritative account of what remains owed. The same logical task may cross several model contexts and processes. Its original requirements and valid observations should remain available even when its current executor disappears.

Goal-oriented requirements engineering has long studied refinement, responsibility, and the relation between lower-level requirements and higher-level goals [1]. Recent agent research extends these concerns to adaptive delegation, explicit verification functions, and dynamic task graphs [2–4]. Durable agent runtimes provide persistence and recovery mechanisms [6–9]. This paper builds on those foundations and concentrates on the consistency boundary between changing execution and accepted completion evidence.

The proposed system keeps the goal contract stable while the execution graph evolves. A deterministic kernel admits graph changes, result acceptance, artifact selection, and success transitions. It retains every accepted obligation, records the subjects to which evidence applies, and prevents stale owners from making authoritative updates. A separate scheduler allocates model and tool capacity. This separation allows a local or cloud model to participate without becoming the authority for its own completion claim.

The paper specifies a state model and transition rules, states conditional correctness properties, proposes a scheduling experiment, and defines an evaluation protocol. These are design contributions at the present stage. The precise mechanisms must still be implemented and evaluated before claims of operational reliability, efficiency, or superiority are justified. The paper makes no priority claim for swarms, durable execution, verification debt, or contract-preserving delegation. Preserving an accepted obligation does not prove that the obligation captures the user's intent, and accepting one snapshot is not the same event as closing the mission.

2 Related work and research position

2.1 Goals and verifiable delegation

Goal-oriented requirements engineering provides a foundation for tracing responsibilities and checking refinements [1]. The relevant distinction is between decomposing work and establishing that the resulting work satisfies the parent goal. A collection of completed subtasks is insufficient when the decomposition is incomplete or its assumptions are inconsistent. This paper therefore retains parent acceptance obligations through decomposition.

Intelligent AI Delegation treats delegation as a transfer of responsibility and authority with monitoring and verification, and proposes decomposing work until outcomes can be checked with available capabilities [2]. VeriMAP operationalizes verification-aware planning through dependency graphs and planner-defined verification functions in Python and natural language [3]. Its published paper has an accompanying public implementation. These works establish that acceptance conditions and intermediate checks are substantive parts of agent coordination.

Goal-Aware Dynamic Multi-Agent Collaboration, or GAD, is especially close to the present design. It grows an execution graph during work and preserves each parent's goal and acceptance conditions. Child completion does not establish parent completion; node goals are checked separately, and successful root verification establishes overall completion [4]. Its described controller also checks dependencies, execution references, budgets, and graph validity. This paper treats GAD as research prior art; the cited preprint is not evidence that a particular public implementation has been independently inspected or benchmarked.

Proof-Carrying Contracts proposes machine-checkable justification for operations that preserve agreed constraints and checks compatibility between delegated contracts [5]. Its formal guarantees depend on the specified model. The authors' status page identifies accompanying mechanization and code as forthcoming at the review date. The present proposal uses an acceptance kernel and permits empirical or judgment-based evidence under declared policies; it does not claim the guarantees of a proof-carrying system merely by storing a certificate.

2.2 Durable runtimes and evidence management

LangGraph documents checkpointed graph state and persistent stores; crash durability depends on the selected storage backend [6]. Microsoft Agent Framework's Durable Extension documents persistent sessions, checkpointed orchestration, and recovery across worker executions [7]. AgentLedger describes leases, fencing, tool governance, evidence archives, and replay [8]. These systems demonstrate that persistence and execution accounting are existing infrastructure rather than inventions of this proposal.

OpenWOP specifies a durable workflow protocol with interruption, replay, idempotency, and child-run dispatch [9]. Its published interoperability evidence should be distinguished from independent ecosystem adoption. Codex Supervisor documents that a product write opens bounded verification debt and that only a later observed successful validation clears it, and that a confirmed completion claim requires a current final-regression certificate [10]. FlowMemory's Obligation Membrane documents a local prototype that rejects missing child obligations, parent closure over open children, and rootfield drift in an agent-commerce setting [11]. These are relevant precedents for evidence freshness and obligation continuity.

Leases are also established distributed-systems machinery [12]. In the proposed runtime, a lease helps decide when an attempt should be reconciled; expiry alone does not prove that an external process or provider request has stopped. The paper distinguishes authority to commit state from the physical lifetime of execution.

2.3 Scope of the proposed contribution

The research target is a specified composition: versioned goal obligations, snapshot-aware evidence acceptance, transactionally enforced graph changes, and recoverable provider handoff within one runtime contract. A second target is the effect of admitting work according to the verification and integration demand it creates. The evaluation must determine whether these choices improve behavior over strong durable and verification-aware baselines. Combining familiar mechanisms does not by itself establish a new scientific result.

Repository documentation is used here to identify overlap and implementation patterns, not to assert independently measured guarantees for other projects. The related-work review is focused rather than exhaustive. A future empirical paper should pin the exact compared implementations, identify any reconstructed methods, and revise the novelty claim if a baseline already provides the tested behavior.

3 System model and assumptions

3.1 Persistent goals and replaceable execution

A mission is a durable unit of user-directed work. It includes the original request, a versioned goal contract, authorized scope, resource policy, and the history of execution and acceptance decisions. An agent is a reasoning context that can be created, suspended, replaced, or retired. A task is a logical work item; an attempt is one execution of that task. The separation prevents a process failure from deleting the work item it was performing.

$$C^v = (O^v, \Gamma^v, \Pi^v) \tag{1}$$

The contract in Equation 1 contains a finite set of accepted obligations, environmental assumptions, and an acceptance policy. Each obligation has a stable identifier and a predicate describing the property to be accepted. The policy specifies which checkers and evidence classes may discharge that obligation. Translating a natural-language request into this contract is a separate, fallible step. The original request remains available for review, and an incomplete contract does not constitute a complete account of user intent.

$$S_r = (C^v, G_r, \sigma_r, E_r, D_r, \Omega_r, X_r) \tag{2}$$

Equation 2 describes a committed runtime revision. Throughout, the superscript identifies the contract version and the subscript identifies the committed runtime revision. In Equation 1, O^v is the obligation set, Γ^v the environmental assumptions, and Π^v the acceptance policy. In Equation 2, G_r is the execution graph, σ_r the selected snapshot, E_r the evidence store, D_r the discharge records, Ω_r the ownership epochs, and X_r the effect ledger.

The execution graph records tasks and dependencies. The selected snapshot identifies the artifacts and relevant environment observations to which current acceptance refers. Evidence and discharge records are separate: storing an observation does not authorize a requirement to be marked satisfied. Ownership epochs identify the attempts currently permitted to commit results. The effect ledger records tool intents, receipts, and uncertain external outcomes.

For a trace property, such as absence of unauthorized operations, the selected snapshot also includes the relevant execution and effect history. Otherwise, the semantic claim is restricted to properties of the represented artifacts and observations. Acceptance is evaluated at a recorded decision time, because evidence can expire even when artifact versions do not change.

Table 1. Persistent entities and their responsibilities

Entity	Responsibility
Mission and contract	Preserve the user goal, scope, accepted obligations, assumptions, and acceptance policy.
Task and dependency graph	Represent logical work and the verified outputs required before dependent work proceeds.
Agent and attempt	Identify the reasoning context, execution instance, provider, and current ownership epoch.
Artifact snapshot	Identify the selected deliverable and relevant input, configuration, and environment versions.
Evidence and discharge	Separate observations from accepted claims about a requirement and snapshot.
Effect and resource ledgers	Track operation outcomes, reservations, release evidence, and recovery obligations.

The dependency graph is acyclic within a committed execution plan. Repair loops create new attempts or explicit alternative routes rather than implicit wait cycles. The communication graph may contain cycles and need not follow parent lineage. Lineage determines responsibility and escalation; dependencies determine readiness; messages carry observations and requests. None substitutes for the authoritative goal contract.

3.2 Evidence applicability

$$e = (\text{claim}, v, \text{subjects}, \text{dependencies}, \text{checker}, \text{outcome}, \text{provenance}) \tag{3}$$

An evidence record binds its claim to a contract version, exact subjects, relevant dependencies, the checker and its version, an outcome, and provenance. In Equation 3, v is that contract version. Subjects may include source files, generated artifacts, test definitions, configuration, or a bounded

observation of external state. Evidence can be authentic as a historical record while being inapplicable to the selected deliverable.

Applicability requires agreement between the record and the current subjects, satisfaction of its relevant assumptions at the decision time, applicable supporting evidence, and acceptance of the checker for that claim. Applicability fails when the selected artifact, a declared dependency, the acceptance policy, or a relevant external condition no longer matches the record. A content hash establishes identity, not correctness. A signature attributes a statement, not its truth. A passing test records the outcome of its assertions on the tested execution. Any broader property depends on the adequacy and soundness of the test and its oracle.

Dependencies include inputs material to the claim. Hashing only the output is insufficient when a changed configuration, test, tool definition, or external state affects the conclusion. Every represented change material to a claim must belong to its dependency closure or conservatively invalidate that claim. Without this assumption, the guarantee concerns declared dependencies only. External-state evidence includes an observation time and validity condition. The cost of conservative revalidation is measured.

3.3 Trust and failure assumptions

The structural model assumes a trusted kernel and durable store with serializable updates or equivalent revision checks. Relevant artifact versions are immutable, and changes to the selected snapshot pass through the kernel. Evidence support graphs are well-founded. Every authoritative result commit checks the current ownership epoch. Time-dependent checks use a declared clock and uncertainty model. Consumers identify contract, snapshot, state revision, and decision time so that historical success is not confused with current acceptance.

Workers may fail, return late, duplicate requests, generate incorrect plans, or propose unsupported completion claims. Transport may duplicate, delay, or reorder messages. The store may recover from a crash according to its documented durability guarantees. Kernel compromise, corrupted trusted storage, and unobserved changes outside the declared dependency model are beyond the stated guarantee. Tool adapters must identify their own effect and reconciliation semantics.

The model distinguishes evidence classes. Deterministic checks and machine-checked proofs can support precisely stated predicates under their assumptions. Model judgments and human reviews may satisfy a declared acceptance policy, but they do not automatically supply logical soundness. Each discharge records its evidence class so that a completion report can express the assurance actually obtained.

4 Runtime architecture

A persistent mission supervisor coordinates a deterministic acceptance kernel, task scheduler, message service, and shared capability broker. Model-backed agents propose work and perform authorized tasks. They do not directly rewrite the accepted contract, claim ownership of completed effects, or mark the root mission successful. All authoritative transitions pass through the same revisioned state boundary.

The scheduler maintains ready work and reservations for each resource class. Local inference capacity, cloud quotas, repository writes, browser sessions, and desktop control have different constraints. A single local inference slot need not serialize independent cloud work. Conversely,

several idle model slots do not authorize concurrent writes to one exclusive resource. Admission and budget reservation must be atomic with respect to competing requests.

A shared capability broker exposes connected tools and skills under mission authorization. Children inherit the authority required for their assigned work, subject to narrower task scope and disabled capabilities. Existing authorization need not be requested again merely because a child acts. Credentials remain in the host. Skills, peer messages, and tool output provide instructions or evidence within their trust level; they cannot enlarge authority.

Tool discovery and execution use the same capability assessment. The catalog distinguishes available, disabled, unsupported, unauthorized, and missing-prerequisite capabilities. Relevant tool schemas and skill content are loaded progressively. This keeps the catalog common without copying every instruction or secret into every context. An independent reviewer may have a deliberately narrower action scope while retaining access to the artifacts it must inspect.

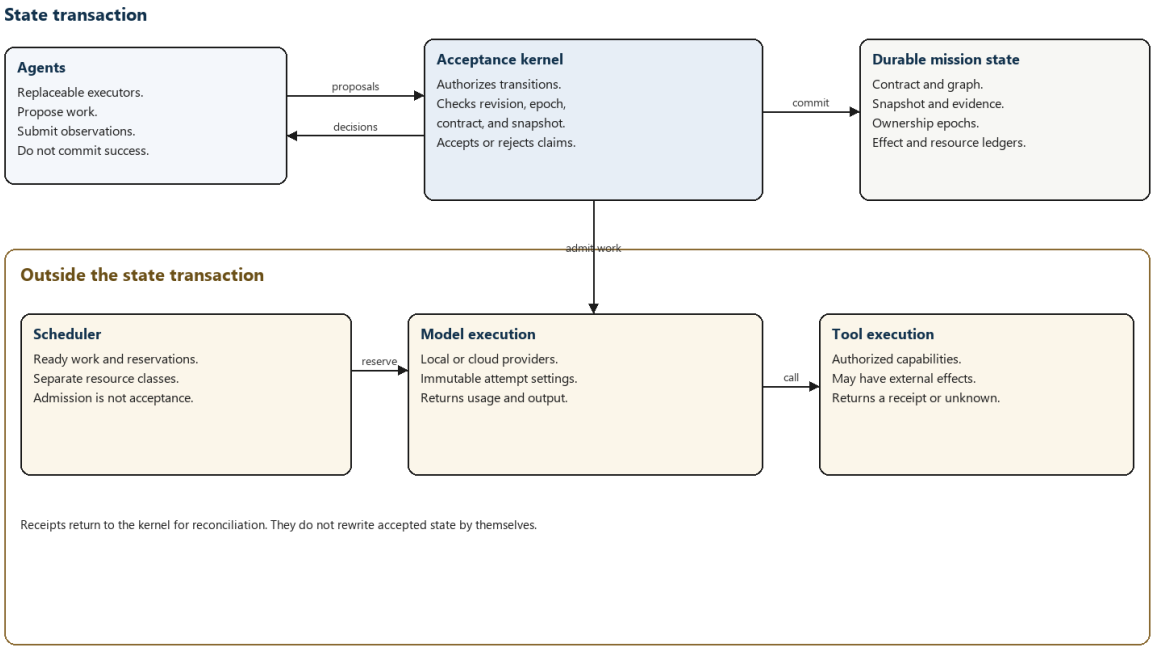


Figure 1. Proposed runtime boundaries. Agents propose work and submit observations. The kernel controls authoritative state transitions and acceptance. Model and tool execution remain outside the state transaction and return receipts for reconciliation. The drawing is a schematic illustration, not an experimental trace. Editable source: figure-1.svg.

Provider adapters declare request protocol, tool-call behavior, structured-output support, reasoning settings, context limits, streaming, cancellation, and usage accounting. Each attempt receives immutable provider configuration. Local or cloud location and Fast or Deep reasoning settings affect model execution; they do not determine whether the runtime supports delegation. A provider unable to support a required capability must be rejected clearly or used through a validated adaptation.

Messages carry mission, task, sender, recipient, correlation, and artifact references. Delivery and acknowledgment are durable and deduplicated. A message can request a state change, but the kernel decides whether to commit it. Decisions that affect dependencies, scope, or acceptance

become structured state. This avoids making one private transcript the only record of a mission-critical decision.

5 Continuity protocol

5.1 Graph mutation and delegation

A decomposition request proposes child tasks, their dependencies, expected artifacts, acceptance conditions, and the parent obligations they support. The kernel validates references, authority, graph validity, and resource commitments. It retains the parent obligation regardless of whether the proposed route is accepted. Completing that route returns control for integration and parent acceptance; it does not discharge the parent automatically.

The operation commits graph changes, applicability invalidations, and dispatch notifications in one transaction. Notifications are published from a transactional outbox. This prevents a committed task from silently losing its dispatch request, but delivery remains potentially duplicated. Consumers therefore use stable command identities and revision checks. A repeated request with changed content under the same identity is a conflict, not a second equivalent command.

Algorithm 1. Apply a proposed transition

```
apply(command, expected_revision, attempt_epoch):
    begin transaction
    require authorized caller and current attempt epoch
    require expected_revision matches committed state
    load contract, affected graph, snapshot, evidence
    validate references, dependencies, and reservations
    preserve every accepted obligation record
    apply mutation and invalidate affected discharges
    append event and durable dispatch notifications
    commit new state revision
    publish notifications with stable command identities
```

Workers wait asynchronously. A parent receives a stable child-task handle, records a checkpoint, and releases its inference reservation before waiting. The scheduler can then run a child on the same single-slot endpoint. Result availability makes the parent eligible to resume; it does not require the parent to retain an active model request. Resource wait cycles and repeated delegation without progress produce explicit diagnostic outcomes.

5.2 Result acceptance and artifact changes

A submitted result identifies its task, attempt, ownership epoch, proposed artifacts, and evidence. Long-running checks execute against a pinned snapshot outside the storage transaction. Before accepting their results, the kernel rechecks the relevant contract, snapshot, policy, and epoch. A changed dependency prevents the old result from becoming a current discharge. The observation remains available for diagnosis or reuse where its original subjects still apply.

Algorithm 2. Accept evidence and evaluate completion

```
accept(result, expected_revision):
    verify result envelope and pinned check subjects
    begin transaction
    require current ownership epoch and state revision
    require contract and relevant snapshot still match
    record evidence with its provenance and evidence class
    check support applicability at the acceptance time
    discharge only claims with applicable accepted support
    if all required obligations are discharged:
        require relevant external effects are reconciled
        record acceptance for this contract and snapshot
        enter closure and fence new dispatch admissions
    otherwise preserve explicit unresolved obligations
    commit revision and publish resulting notifications
```

Artifact selection is an acceptance-relevant transition. If a selected file, dependency, or configuration changes, the same transaction marks affected discharges inapplicable and updates the current acceptance projection. Historical successful snapshots remain in the record. Selecting a revised deliverable requires a fresh acceptance decision. The runtime must not expose a new artifact together with a success label inherited from an old snapshot.

Freshness is also checked when current acceptance is queried or used to admit an action, and at terminal completion. Expiration can invalidate support without a new artifact revision. Timed invalidation events may maintain a cached projection, but they do not replace the applicability check at the decision point. Historical acceptance remains a decision about its pinned snapshot at its recorded time, not a permanent assertion about external truth.

Snapshot acceptance and terminal mission completion are separate events. An accepted snapshot initiates closure, which fences new task and tool dispatch admissions, settles or cancels outstanding descendants, and reconciles relevant effects and resource releases. Previously admitted operations may still begin or complete. A late response cannot reopen admission or overwrite accepted state through an expired authority. Material changes or evidence expiry during settlement require renewed acceptance. Unknown effects or occupied resources keep closure unresolved. The evaluation records acceptance and settlement timestamps separately.

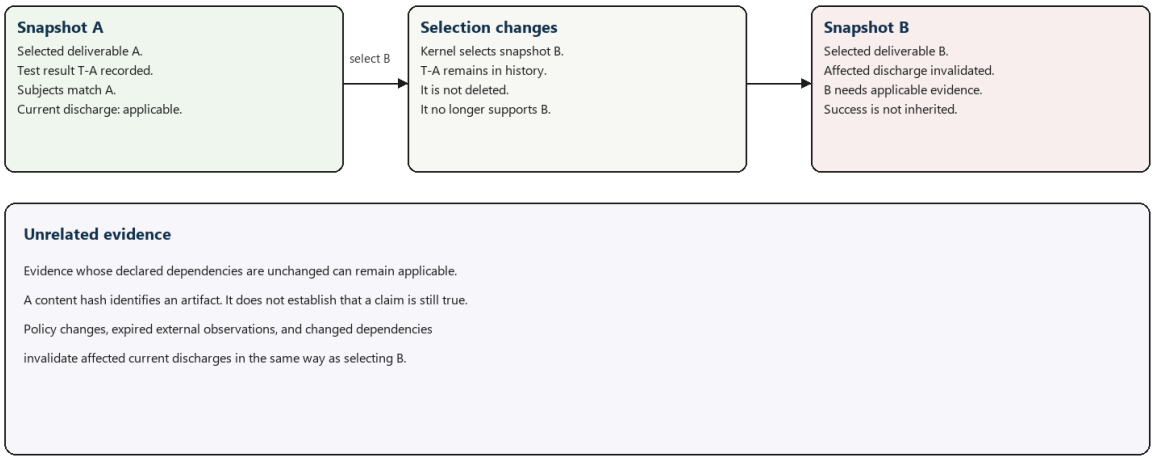


Figure 2. Evidence survives as history while applicability changes. A test result for snapshot A supports A. Selecting snapshot B invalidates the affected current discharge until B receives applicable evidence. Unrelated evidence can remain usable when its dependencies are unchanged. The drawing is a schematic illustration, not an experimental trace. Editable source: figure-2.svg.

5.3 Failure recovery and external effects

Task identity survives attempt failure. Recovery inspects the durable task, evidence, effect, and resource records before deciding whether to resume, reassign, replan, or report a blocker. Ownership moves through a new epoch. A late result from an earlier epoch cannot authoritatively discharge an obligation. It may still be retained as an observation for controlled reconciliation.

A tool operation has a stable logical identity, argument hash, recorded intent, dispatch status, and result or uncertain outcome. Internal deduplication prevents repeated messages from becoming repeated logical commands. External exactly-once effects require support from the downstream system or an equivalent end-to-end mechanism. If an operation may have succeeded before its acknowledgment was lost, a blind retry can duplicate its effect. The adapter must reconcile, use supported idempotency, or retain an explicit unknown outcome.

A lease expiration revokes or challenges authority under the runtime policy; it does not establish that physical execution has ended. Resource capacity remains occupied or uncertain until the relevant release receipt, confirmed cancellation, process observation, or other resource-specific evidence supports reuse. Fencing checks at the ledger do not stop an old process from acting on an unfenced external tool. Enforcement must also exist at the effect boundary where the claim requires it.

Cancellation prevents new dispatch admissions after its authoritative transition. Operations admitted earlier may still begin or complete, and their outcomes must be reconciled. A transactional outbox does not remove the interval between admission and an external request. A user stop remains durable across restarts. Application detachment or process loss has a separate state so that it does not silently become either a cancellation or permission for unbounded continuation.

5.4 Provider handoff and contract amendment

Provider handoff creates a new attempt from a portable checkpoint. The checkpoint contains task and contract identities, selected artifact references, relevant observations, completed tool receipts,

message position, capability requirements, and unresolved effects. It resumes the logical task; opaque provider-internal reasoning or session state is not assumed transferable. The new provider is admitted only if it meets the task's capability and destination policy.

Existing artifact evidence need not become invalid solely because the model changes. Applicability follows the claim's subjects and assumptions. A judgment whose acceptance policy depends on the reviewer model may require reassessment; a recorded deterministic test over unchanged artifacts may remain applicable. These dependencies should be explicit rather than inferred from the provider label.

A scope amendment creates a new accepted contract version with an authorized relationship to the preceding version. The old obligations and their outcomes remain in history. Evidence is reassessed for the new contract. Removing a requirement cannot retroactively establish success under the old contract. Changes to execution strategy ordinarily require no amendment because tasks and agents are not the accepted goal itself.

6 Conditional correctness properties

6.1 Structural continuity

For a selected snapshot and decision time, an obligation is discharged only when the active policy admits applicable support. The starred discharge set in Equation 4 contains the identifiers of those obligations, projected from the discharge records. Its complement is the unresolved set, including pending, blocked, failed-attempt, and cancelled-but-unfulfilled work. The projection partitions the obligations of one unchanged contract version.

$$O^{v} = U_{r}(\tau) \uplus D_{r}^{*}(\tau) \tag{4}$$

Proposition 1. Under the storage, ownership, dependency, and mediation assumptions in Section 3, every reachable committed state retains all obligations of the active contract. A stale ownership epoch cannot commit a discharge, and current success implies applicable discharge of every required obligation.

$$\text{Success}(C^{v}, \sigma_{r}, \tau) \Rightarrow \forall o \in O^{v}, \text{Discharged}(o, \sigma_{r}, E_{r}, \tau) \tag{5}$$

In Equation 5, τ is the decision time at the committed revision r. Success here denotes acceptance of the stated contract and snapshot.

Proof sketch. Initially, every accepted obligation is recorded and unresolved. Delegation and reassignment preserve obligations. Failure transitions modify attempts without deleting their tasks or requirements. Snapshot changes atomically invalidate affected discharges. Result acceptance checks the current epoch and support applicability at its decision time. The sole success transition checks every required obligation against one compatible selected snapshot; later current-state queries re-evaluate time-dependent applicability. Induction over committed transitions and the decision rule establishes the invariant. Messages alone cannot alter authoritative state. A contract amendment starts a separate version and preserves the earlier history.

Terminal mission completion additionally requires the closure conditions in Section 5.2. This proposition is a conditional property of the specified transition system. It is not a claim that an implementation has been model checked, that every dependency has been captured, or that a checker's positive judgment is substantively true. Any implementation path that bypasses the kernel invalidates the induction premise.

6.2 Semantic soundness under stronger assumptions

Let an obligation predicate describe its intended property over a snapshot and decision time. Suppose accepted leaf evidence is sound for that predicate. Suppose compositional discharge uses a sound refinement with satisfied environmental assumptions, consistent premises, and compatible subjects. In Equation 6, the assumption term denotes the conjunction of all contract assumptions evaluated at that snapshot and time; the entailment must hold over admissible states. Equations 6 and 7 state that entailment for a selected snapshot σ and decision time τ. They do not add a second revision index. This is the established goal-refinement problem [1].

$$\Gamma^{v}(\sigma,\tau) \wedge \forall j \in \{1,\dots,k\}, \varphi_{c_j}(\sigma,\tau) \vDash \varphi_{p}(\sigma,\tau) \tag{6}$$

Proposition 2. Under these stronger assumptions and Proposition 1, mission success implies that every required predicate of the accepted contract holds over the selected snapshot.

$$\text{Success}(C^{v},\sigma,\tau) \Rightarrow \forall o \in O^{v}, \varphi_{o}(\sigma,\tau) \tag{7}$$

Proof sketch. Induct over the well-founded evidence support graph. Sound leaf certificates establish their predicates. Each justified refinement establishes its parent predicate from compatible children and satisfied assumptions. Structural continuity ensures that success contains applicable support for every required obligation. The conclusion follows for the accepted contract, not for unspecified aspects of the original natural-language request.

An LLM-generated coverage explanation does not prove the entailment in Equation 6. A practical runtime can require a direct parent-level acceptance check where sound composition is unavailable. When the accepted checker is heuristic, the runtime guarantees traceable application of that policy rather than semantic truth. This distinction must remain visible in both reports and benchmark labels.

6.3 Progress and impossibility boundaries

The safety properties do not establish eventual completion. Progress additionally requires a feasible goal, adequate resources, available tools, suitable models, and fair scheduling. An agent cannot be guaranteed to solve an arbitrary problem by receiving additional attempts. Budget exhaustion, persistent external uncertainty, or an unsatisfied prerequisite must remain a truthful non-success outcome.

Four counterexamples clarify the boundary. A decomposition may omit authorization while completing interface and storage tasks. A test may pass before a configuration change makes its conclusion inapplicable. Two valid child certificates may concern incompatible branches. A perfectly authenticated reviewer judgment may be wrong. The kernel preserves omitted

obligations and rejects recorded dependency or snapshot conflicts. Detecting a semantic omission additionally depends on an adequate specification and acceptance checks.

7 Admission based on verification demand

Dynamic population control should account for the work created by an assignment after its initial execution. Producing an artifact often creates a need for review, integration, and revalidation. Admitting workers using only free inference slots can consume the budget required to establish that the mission succeeded. The proposed policy estimates and reserves these downstream costs before admitting discretionary work.

$$\hat{b}_i = \hat{c}_i^{\text{exec}} + \hat{c}_i^{\text{verify}} + \hat{c}_i^{\text{integrate}} \tag{8}$$

Equation 8 is a scheduling estimate expressed in one declared accounting unit, such as model tokens or normalized compute time. Different resource classes require separate reservations; dollars, tokens, and GPU seconds must not be added without an explicit conversion model. Actual usage replaces estimates as evidence arrives, and the scheduler updates remaining reservations when a task decomposes.

$$\sum_{i \in I_r} \hat{b}_{i,r}^{\text{remaining}} + b_{\text{finish},r}^{\text{remaining}} \leq B_{\text{remaining},r} \tag{9}$$

Equation 9 constrains residual reservations at the current revision. The indexed set contains outstanding commitments, including verification after a worker exits. Consumed work is charged to actual usage and removed from remaining estimates. Decomposition transfers or revises reservations rather than charging parent and child independently for the same work. Shared checks are charged once; invalidation that creates new work creates a new residual estimate. The finalization reserve covers remaining root acceptance, closure, and recovery. This is not a bound on total agents used over the mission.

A second control observes unfinished verification demand. When it exceeds a configured allowance, discretionary fan-out is delayed while checks and recovery receive capacity. This is an application of established backpressure and resource reservation ideas, not a new queueing principle. The policy must preserve work needed to discharge the backlog: verification or prerequisite tasks cannot be blocked merely because the backlog is high.

Underestimated costs can exhaust reserves. Overestimated costs can suppress useful parallelism. Strict verifier priority can starve execution, while strict worker priority can starve verification. The policy therefore needs bounded fairness, safe checkpoint suspension, explicit handling of infeasible commitments, and telemetry that separates estimate error from model failure. Evaluation should compare against work-conserving scheduling and fixed verification reserves, rather than only a weak scheduler with no protection for completion.

The scheduling hypothesis is that accounting for downstream verification improves verified mission completion per budget or reduces completion latency at comparable quality. The acceptance kernel and scheduler are separate experimental factors. If both compared systems enforce the same success invariant, differences in productivity should not be misattributed to a stronger completion rule.

8 Implementation setting in Pegasus

Pegasus is the proposed implementation setting. The observations in this section were read from `integrations/agent_control.py` and `integrations/adaptive_tools.py` at development commit `c581ed9bf079c5923b34004d02ad82dbd183fb82`. On 7 November 2026 that snapshot was compared with main at `6e389d3fe43315918ef29e515d42bcc643acd6fb` [13, 14]. The same two commit objects, and the cited controller and broker lines at the development commit, were re-read on 8 November 2026. This re-reading did not produce a new diff against main. These are pinned source observations. They are not claims about a running installation, a newer tree, or a completed implementation of this paper.

The inspected controller counted four spawns in a parent turn and then refused another spawn. A slot was returned for a timed-out, failed, stopped, interrupted, or incomplete run, and was not returned for a finished run. Queue admission checked whether any run occupied the shared model slot, rather than partitioning capacity by endpoint. A further continuation was refused when the recorded continuation index reached 2. Ownership checks used the parent process identifier and chat identifier; provider release also checked the execution identifier and continuation index [13]. These policies are reasonable to treat as existing constraints to migrate, not as evidence of a general persistent mission supervisor.

The Adaptive tool broker in `integrations/adaptive_tools.py` at the same development commit mediated authorized leaf operations, recorded uncertain outcomes, and refused nested dispatch and delegation wrappers including `pegasus_agents`, `delegate_task`, and related tool-call names [14]. The appropriate extension is a first-class asynchronous child-task request that retains identity and authority. Simply removing a blocked-tool name would not provide checkpoint waiting, resource release, or durable parent recovery.

Implementation should begin with versioned mission and acceptance records, then integrate task ownership and resource scheduling, then route recursive workers through the common capability boundary. Existing exact release receipts and idempotent coordination should be preserved or replaced by equivalents with explicit migration evidence. Active scheduling must use authoritative indexed records rather than bounded recent-history scans. Native request entry points must exercise the same lifecycle before the design is described as integrated.

Implementation status should be reported per capability using separate labels for proposed, present in source, integration-tested, native-validated, and operationally observed behavior. The current paper supplies the proposed contract and evaluation design. It does not supply empirical swarm results, a release certification, or evidence that a particular binary implements these properties.

9 Evaluation protocol

9.1 Questions and experimental layers

The primary correctness question is whether the kernel prevents silent obligation loss and inapplicable-evidence acceptance under the specified fault model. The productivity question is whether the resulting system completes useful missions under comparable resource budgets. A separate scheduling question asks whether verification-aware admission improves those outcomes beyond the kernel alone. The evaluation must keep these questions distinct.

The first layer uses scripted workers, deterministic tool fixtures, controlled clocks, and bounded state-machine exploration. It generates graph changes and execution interleavings without relying on model behavior. A counterexample trace that loses an accepted obligation or accepts stale evidence refutes the structural implementation claim. Passing a finite suite establishes coverage of that suite; it does not prove correctness over every possible interleaving.

Protocol experiments supply the same independently checked contract to every compared system. A separate end-to-end condition evaluates extraction of that contract from the original request. Held-out checks test the declared requirements rather than introduce undisclosed goals. This separates requirements-extraction errors from continuity failures and prevents a shorter, easier contract from appearing to improve reliability.

The second layer uses the actual native application, host, tool broker, persistence backend, and provider adapters. It records source commit, dependency pins, build identity, runtime configuration, operating environment, model identifier, reasoning settings, and endpoint. A mocked model or passing module test is insufficient evidence that the installed application follows the proposed lifecycle.

9.2 Fault scenarios and independent oracles

Table 2. Required fault scenarios and observable outcomes

Scenario	Required observation
Crash around commit or dispatch	Committed work remains discoverable; duplicated notification does not duplicate a logical task.
Lost tool acknowledgment	The external outcome is reconciled or remains explicitly unknown before a potentially duplicating retry.
Expired owner returns	A late attempt cannot commit an authoritative discharge under a superseded epoch.
Artifact changes during review	A result for the old snapshot cannot authorize acceptance of the revised deliverable.
Incomplete decomposition	An omitted root requirement remains unresolved despite successful child tasks.
Incompatible child artifacts	Separate passing results do not authorize an unchecked integrated snapshot.
Provider handoff	The logical task and usable evidence survive; opaque reasoning state and uncertain effects are not assumed portable.
Single-slot nested delegation	A waiting parent releases inference capacity and does not deadlock its child.
Cancellation race	New dispatch admissions are prevented; previously admitted operations and their effects remain accounted for.
Late response during closure	No new dispatch admission or stale commit occurs; outstanding effects and resources are settled before terminal completion.
Verification overload	The scheduler preserves verification and finalization opportunities and reports infeasible budgets honestly.

Inject faults at named semantic boundaries rather than arbitrary wall-clock offsets: before persistence, after an effect but before its receipt, during epoch transfer, or between artifact selection and evidence acceptance. Equivalent fault opportunities should be aligned across

baselines. Observe duplicate effects at the tool fixture or external-system boundary rather than relying on the runtime's own success log.

Use predefined repository tasks, multi-artifact tasks, and bounded research tasks. Include missions with useful parallelism and missions where delegation adds overhead. Final behavioral tests and requirement checklists should be held out from tuning. For judgments that require humans, blind reviewers to the system where practical and document disagreements. Model review may be one signal, but it should not be the sole independent oracle for a claim about model reliability.

9.3 Baselines and ablations

Table 3. Main comparisons and the mechanism isolated

Variant	Purpose
Unchanged verified Pegasus build	Measure product improvement against a pinned and independently exercised native baseline.
Single agent with equivalent capabilities	Establish whether delegation adds value under the same aggregate budget.
Fixed team with durable execution	Separate dynamic organization from ordinary parallel work.
Dynamic graph with root verification	Provide a strong baseline that already checks the complete goal.
Dynamic graph plus continuity kernel	Isolate obligation retention, evidence applicability, and ownership enforcement.
Kernel plus verification-aware admission	Isolate the effect of downstream-work reservations and scheduling.

Use actual related implementations where their capabilities and experimental interfaces permit a fair comparison. A reconstruction of a paper must be labeled as such. Give baselines comparable tools, skill access, model settings, prompt tuning effort, and acceptance requirements. An in-memory configuration is not a fair proxy for a durable backend when crash recovery is the tested property.

Hold total mission budgets constant across parents, children, reviewers, and failed attempts. Equal per-agent budgets would favor systems that spawn more agents. An outcome obtained with a larger budget, under a weaker contract, or by a metric that counts only reported successes is not an improvement under this protocol. Evaluate local, cloud, and mixed-provider cases separately, and run within-model comparisons before attributing an effect to heterogeneous routing. For heterogeneous providers, use a declared monetary or compute accounting convention and stopping rule; raw token counts across tokenizers are not equivalent budgets. Include work-conserving scheduling and fixed verification reserves as scheduling baselines.

Ablate obligation retention, evidence version binding, invalidation propagation, and ownership fencing individually. Then test interactions, particularly artifact revision combined with a late stale owner. For the scheduling policy, ablate downstream-cost estimation and the finalization reserve separately. Keep mandatory terminal verification identical when comparing admission policies.

9.4 Metrics and statistical analysis

The primary mission-level outcome is correct completion: an independently accepted deliverable accompanied by an appropriate completion report and operational settlement. False success is a reported success that fails the independent acceptance or settlement procedure. Report false-success frequency over all attempted missions and the conditional error among reported successes; the latter is undefined when a system never reports success. Also report correct completion, unresolved outcomes, blocked runs, and budget exhaustion so that a system that always holds cannot appear effective.

Protocol metrics include lost obligation records, stale-evidence acceptance, stale-owner commits, duplicate logical commands, independently observed duplicate effects, and violations of delegated scope. Recovery metrics include time to resume useful work, re-executed verified work, retained applicable evidence, and permanently unresolved effects. A zero observed violation count must state the number and diversity of tested opportunities.

Measure end-to-end elapsed time, including queueing, checking, retries, recovery, and closure. Record human-wait time separately while retaining it in the user-facing duration. Sum tokens and tool calls across all participants and attempts, including billed, cached, and reasoning usage where reported. Identify estimated or unavailable usage. Report cloud charges from billing records or dated price assumptions, and local inference time and utilization under an explicit cost convention. Check sensitivity to local-cost assumptions when they affect the conclusion. Parallel worker durations are not wall-clock mission latency.

Use a common observation horizon and a prespecified fixed-horizon or time-to-success analysis that retains incomplete runs. Comparing latency only among successful missions can reward a system that solves only easy tasks. Report false holds: missions whose artifact and settlement criteria are independently satisfied but which the runtime does not accept as complete within the horizon. Report cost per attempted mission and total evaluation cost divided by independently successful missions, including spending on unsuccessful attempts.

Select tasks, budgets, outcomes, and stopping rules before final evaluation. Use pilot variance to choose replication and a meaningful detectable effect rather than selecting a convenient sample size. Randomize run order and use paired task and fault conditions. Estimate uncertainty at the task level, accounting for correlated repetitions. Report effect sizes, intervals, distributions, exclusions, and all failure categories. Tuning tasks and final evaluation tasks must remain separate.

A practical acceptance criterion should constrain both improvement and overhead. For example, the kernel must reduce a specified failure category without exceeding a preregistered loss in correct completion or resource use. Numeric thresholds must be selected for the target application before the final runs. They are not supplied here as measured results. Negative findings, including equivalence with a strong baseline, remain informative.

10 Limitations and research risks

The largest semantic risk is specification error. A complete ledger of incomplete requirements can still authorize an incomplete product. Preserve the original request, use direct root acceptance, and evaluate requirements extraction separately. A user-authorized amendment should be explicit, but ordinary execution choices should not require repeated confirmation. Excessive approval friction is itself a measurable cost.

Evidence dependency tracking may be incomplete or overly conservative. Missing dependencies admit stale conclusions; excessive invalidation wastes verified work. External systems may change without a stable version identifier. The applicability policy must state what is observed, how long the observation is usable, and what uncertainty remains. This paper does not solve arbitrary world-state verification.

Provider adapters can normalize interfaces without equalizing model competence. A smaller model may fail to decompose a task that a stronger model handles directly. Cross-provider continuation may omit tacit context despite a well-formed checkpoint. These are empirical risks that require task-level measurements rather than a claim that all models become interchangeable in quality.

A central acceptance kernel can become a bottleneck or a single point of failure if deployed without suitable persistence and recovery. The initial single-host design intentionally makes authoritative transitions easy to inspect. Scaling the storage and execution topology requires a separately specified consistency model. Durable logs also create storage, retention, and sensitive-data obligations; retain the evidence required for a claim while applying the mission's data policy.

Verification-aware admission relies on uncertain cost estimates and may reduce throughput on tasks with cheap or unnecessary checks. Mandatory controls must be separated from experimental heuristics. The runtime should expose why work was queued or held, and the benchmark must measure starvation, idle capacity, and the cost of conservative decisions. No theoretical productivity improvement follows from adding reservations alone.

11 Conclusion

Adaptive swarms need a durable relationship between the goal they accepted, the work they perform, and the evidence supporting completion. This paper specifies that relationship through versioned obligations, separate task and attempt identities, snapshot-aware evidence, and mediated state transitions. Under explicit assumptions, the transition model preserves structural continuity; semantic soundness requires stronger assumptions about specifications, checkers, and refinements.

Pegasus offers a concrete setting for implementing the contract across local and cloud execution. The proposed evaluation separates runtime correctness from model capability and distinguishes the effects of continuity enforcement from verification-aware scheduling. The next research result is an executable implementation with reproducible counterexample tests, fair baselines, and measured completion and cost outcomes. The contribution should be judged by those results and by the precision of the claims they support.

Appendix A. Minimum reproducibility record

A reproducible experiment should publish a manifest linking the source and build identity to the exact task set, accepted contract, provider settings, tool contracts, fault schedule, and outcome procedure. Record dependency versions, model identifiers, reasoning profiles, budget units, local hardware, operating environment, storage configuration, and any human interventions. Provider aliases that change over time should be identified as such.

Each run should retain stable mission, task, attempt, operation, and evidence identifiers; contract and snapshot revisions; ownership epochs; event ordering; actual resource use; and the final oracle

result. Share redacted traces and deterministic fixtures where sensitive inputs cannot be released. Redaction must preserve the relationships needed to reproduce the tested invariant, and any resulting limitation should be documented.

At minimum, release the transition schemas, migration instructions, fault-injection runner, independent outcome checks, baseline configurations, analysis code, and a claims-to-evidence index. Mark hypothetical examples, scripted protocol tests, live-model trials, and operational observations distinctly. Pin external projects and record the access date for living documentation. This paper's worked diagrams are conceptual illustrations, not experimental traces.

References

[1] Axel van Lamsweerde. 2001. Goal-Oriented Requirements Engineering: A Guided Tour. *Proceedings of the Fifth IEEE International Symposium on Requirements Engineering*. doi:10.1109/ISRE.2001.948567. https://doi.org/10.1109/ISRE.2001.948567

[2] Nenad Tomašev, Matija Franklin, and Simon Osindero. 2026. Intelligent AI Delegation. arXiv:2602.11865v1, 12 February 2026. https://arxiv.org/abs/2602.11865

[3] Tianyang Xu, Dan Zhang, Kushan Mitra, and Estevam Hruschka. 2026. Verification-Aware Planning for Multi-Agent Systems. *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2026)*, Volume 1, Long Papers, pp. 7528–7546, Rabat. doi:10.18653/v1/2026.eacl-long.353. https://aclanthology.org/2026.eacl-long.353/ Code: https://github.com/megagonlabs/veriMAP Preprint: arXiv:2510.17109, 20 October 2025. https://arxiv.org/abs/2510.17109

[4] Xu Shen, Lingyong Yan, Shuaiqiang Wang, Jianmin Wu, Zhikai Chen, Jiliang Tang, and Jie Ren. 2026. Goal-Aware Dynamic Multi-Agent Collaboration for Long-Horizon Tasks. arXiv:2610.39193v1, 29 October 2026. https://arxiv.org/abs/2610.39193

[5] Carlo Ciliberto, Alessandro Rudi, and Enrico Tassi. 2026. Proof-Carrying Contracts: Commitments AI Agents Cannot Break. arXiv:2610.28484v1, 20 October 2026. https://arxiv.org/abs/2610.28484 The authors' program page identified accompanying mechanization and code as forthcoming when checked on 7 November 2026. https://www.di.ens.fr/~rudi/programs/pc2.html

[6] LangChain. n.d. Persistence. LangGraph documentation. Accessed 7 November 2026. https://docs.langchain.com/oss/python/langgraph/persistence

[7] Microsoft. n.d. Durable Extension. Microsoft Agent Framework documentation. Accessed 7 November 2026. https://learn.microsoft.com/en-us/agent-framework/hosting/durable-extension

[8] yaogdu and AgentLedger contributors. n.d. AgentLedger: Durable execution and reliability runtime for production AI agents. Source repository and documentation. Accessed 7 November 2026. https://github.com/yaogdu/AgentLedger

[9] OpenWOP contributors. n.d. OpenWOP: Open Workflow Orchestration Protocol. Specification and source repository. Accessed 7 November 2026. https://github.com/openwop/openwop https://openwop.dev

[10] yimengbenxin. n.d. Codex Supervisor. Source repository `codex-goal-supervisor`, commit 87844d8396f4af73c2be2df6728131f07be62a73. Accessed 7 November 2026. That revision was reconfirmed on 8 November 2026. https://github.com/yimengbenxin/codex-goal-supervisor

[11] FlowmemoryAI. n.d. FlowMemory Uniswap v4 Hooks: Obligation Membrane. Local prototype documentation. Accessed 7 November 2026. https://github.com/FlowmemoryAI/flowmemory-uniswap-v4-hooks/blob/main/docs/OBLIGATION_MEMBRANE.md

[12] Cary G. Gray and David R. Cheriton. 1989. Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency. *Proceedings of the Twelfth ACM Symposium on Operating Systems Principles*, pp. 202–210. doi:10.1145/74850.74870. https://doi.org/10.1145/74850.74870

[13] PEGASUS contributors. 2026. Agent controller, `integrations/agent_control.py`. Development snapshot c581ed9bf079c5923b34004d02ad82dbd183fb82, compared on 7 November 2026 with main 6e389d3fe43315918ef29e515d42bcc643acd6fb. Inspected 7 November 2026. The same commit objects were re-read on 8 November 2026. https://github.com/jacksonjp0311-gif/PEGASUS/commit/c581ed9bf079c5923b34004d02ad82dbd183fb82 https://github.com/jacksonjp0311-gif/PEGASUS/commit/6e389d3fe43315918ef29e515d42bcc643acd6fb

[14] PEGASUS contributors. 2026. Adaptive tool broker, `integrations/adaptive_tools.py`. Development snapshot c581ed9bf079c5923b34004d02ad82dbd183fb82. Inspected 7 November 2026. https://github.com/jacksonjp0311-gif/PEGASUS/commit/c581ed9bf079c5923b34004d02ad82dbd183fb82